SPPH 381H: Health Data Science - AI and Knowledge Translation

Course Outline - Sept-Dec 2026

School of Population and Public Health, University of British Columbia

2026-08-31

Acknowledgement

UBC's Point Grey Campus is located on the traditional, ancestral, and unceded territory of the xʷməθkʷəyʷəm (Musqueam) people. The land it is situated on has always been a place of learning for the Musqueam people, who for millennia have passed on their culture, history, and traditions from one generation to the next on this site.

Course Information

Course Title:	Health Data Science: AI and Knowledge Translation
Credit Value:	3
Format:	Integrated Seminar & Hands-on Activities (1 session/week, 3 hours)
Time:	Tue 11:00 AM - 2:00 PM (Sept-Dec 2026)
Room:	Room posted in Canvas (in-person)

Course at a Glance

Core workspace	GitHub Codespaces (GitHub, Quarto, & VS Code in the browser)
Core language	R, Quarto, and reproducible project workflows
Secondary exposure	Short Python examples for polyglot awareness; optional extension pathway
Main project product	Reproducible Quarto report + required dashboard-style KT product + auditable repository
Primary assessment logic	Foundational gates, best-4 weekly assignments, staged project milestones, presentation, final portfolio

Prerequisites

None required. This course adopts a "Zero to AI Co-Pilot" pedagogy. We welcome students from all disciplines. You do not need prior experience with R, Python, statistics, or programming. We teach foundational concepts and tools from the ground up.

What "no prerequisites" means in practice

This is a beginner-accessible course, not a no-work course. You will learn a substantial but carefully scaffolded toolkit at a steady pace: R, GitHub Codespaces, Quarto, Git, reproducible workflows, AI-assisted coding, basic Python awareness, and dashboard-style knowledge translation. Because coding is a cumulative skill, the weekly assignments are carefully scaffolded to serve as your core practice. If you are new to programming, completing these assignments is how you will steadily build your foundational skills during the first month. Comfort with web browsers, basic file management, and willingness to troubleshoot are assumed; everything else is taught and supported through templates, in-class studio time, peer debugging, and TA support.

Primary Language: R, with Python Awareness

This course uses **R** as the primary programming language for all core graded work. R is widely used in epidemiology, biostatistics, public health, and reproducible research, and the tidyverse ecosystem (`dplyr`, `ggplot2`, `plotly`, Quarto, and dashboard-style reporting tools) provides a coherent pipeline from data import to publication.

The course remains **polyglot-aware** without becoming a full second programming course. Students will read and run short **Python/pandas** examples in GitHub Codespaces, compare Python and R solutions with AI assistance, and learn when Python may be the more appropriate tool. A Python-flavoured option may be offered for one activity or project extension, but students can complete all required outcomes in R.

Students will create a small dashboard-style knowledge translation product as part of the portfolio. The required core will focus on approaches that work reliably in GitHub Codespaces and Quarto-based publishing. Shiny, Shinylive, and AI-generated static HTML/JavaScript dashboards may be demonstrated as optional or advanced deployment pathways rather than required technologies for every student.

Computing Environment: Fork-Based Workflow

All supported coding takes place in **GitHub Codespaces** - a cloud computer accessible from any modern web browser. No local installation is required. Students **fork** a course template repository into their own GitHub account, then launch Codespaces from their fork.

Tools required for graded work will be selected and tested for use in Codespaces. Optional demonstrations may show additional deployment pathways, but students will not be penalized if an advanced demo technology is unsuitable for their device or project.

Contacts

Role:	Course Instructor
Name:	Dr. M. Ehsan Karim
Email:	ehsan.karim@ubc.ca
Office:	Appointments by request
Hours:	Weekly instructor and TA support times will be posted on Canvas

Typical response time: within 48 hours on weekdays.

Other Instructional Staff: TBA. To communicate with the computing TA, attend weekly classes and specified drop-in hours (as announced in Canvas).

Instructor Biographical Statement

Dr. M. Ehsan Karim is an Associate Professor in Health Data Science at the UBC School of Population and Public Health (SPPH), a Scientist at the Centre for Advancing Health Outcomes, an associate member of the Department of Statistics (UBC), and a previous Michael Smith Foundation for Health Research (MSFHR) Scholar. He obtained his PhD in Statistics from the University of British Columbia and completed his postgraduate training in the Department of Epidemiology, Biostatistics, and Occupational Health at McGill University. His current research focuses on causal inference, real-world observational data analyses, and applications of machine learning approaches in epidemiologic studies.

Course Description

This interdisciplinary course bridges traditional epidemiology, modern data science, responsible AI use, and knowledge translation. Using a "Zero to AI Co-Pilot" pedagogy, students learn how to work with health-related data in a cloud-based environment, use generative AI as a coding assistant, and critically audit AI-generated code and outputs. The course is beginner-accessible and assumes no prior coding or statistics background, while still requiring steady hands-on practice.

The curriculum emphasizes reproducible research and the "last mile" of knowledge translation: transforming real-world health data into clear, auditable, and audience-appropriate products. By

the end of the course, students will have built a portfolio that includes documented code, a reproducible Quarto report, clear visualizations, a dashboard-style knowledge translation product, and a reflection on AI use and auditability.

Course Structure

This course integrates theoretical foundations with the practical realities of modern health data science. Through short seminars, guided setup time, hands-on studio work, and structured peer support, you will move from basic data literacy to reproducible, portfolio-ready communication products: reports, dashboards, visual stories, and project repositories that can be audited by others.

Course Pillars

1. **The Full Data Lifecycle:** From ethical data access and cleaning to secure cloud computing and final publication.
2. **Reproducible Science:** Git-based workflows, a shared devcontainer environment, explicit dependency notes, and literate programming in Quarto - ensuring analyses are transparent, shareable, and robust.
3. **AI-Augmented Workflows:** Ethically leveraging LLMs as coding co-pilots to debug, refactor, and translate code, while auditing AI-generated outputs for coding errors, hallucinated claims, security risks, health bias, and reproducibility failures.
4. **Knowledge Translation (KT):** The "last mile" of data science - transforming results into dashboard-style products, visual stories, plain-language summaries, and policy-ready reports. Shiny and static web approaches are introduced as optional or advanced pathways rather than assumed for every project.
5. **Real-World Health Data:** Hands-on activities using messy, real-world datasets (e.g., NHANES, Canadian PUMFs) rather than toy examples.

Session Structure (3-Hour Block)

Each weekly session generally follows the following segments:

Segment	Duration	Description
Seminar and demonstration	45 min	Concepts, examples, and live coding by the instructor
Guided setup	20 min	Everyone completes the first step of the activity together; TAs confirm students are unblocked

Segment	Duration	Description
Break / technical reset	10 min	Short break and time to resolve login, rendering, or package issues
Hands-on studio	90 min	Independent or group work on the weekly activity; TA circulation and peer debugging
Wrap-up and next steps	15 min	Expected output demo, common errors, submission expectations, and preview of next week

Peer Support: Debugging Partners

To scale support with high enrollment, each student is assigned a **debugging partner**. Before raising a hand for TA help, try your partner first. This builds collaboration skills and reduces wait times for everyone.

Asynchronous Onboarding (Week 0)

Due before the first class session (Week 1).

A short Canvas module to complete before the term begins:

1. Create a **GitHub account** at github.com using your UBC email.
2. Choose a **professional username** (it appears on your GitHub profile, and on your portfolio if you later choose to make it public).
3. Watch a **5-minute orientation video** (posted on Canvas) introducing the course workflow.
4. Complete the **pre-course survey** (anonymous; helps the teaching team calibrate to the cohort's baseline experience).

This takes approximately 15-20 minutes and ensures Week 1 can focus entirely on health data concepts rather than account setup.

Schedule of Topics

Week	Seminar Topics	Activities	Deadlines
		Part 1: Foundations & Workflows	
0 (Async)	-	Create GitHub account; watch orientation video; pre-course survey.	Complete before first class

(continued)

Week	Seminar Topics	Activities	Deadlines
1 (Sept 15)	Health Data, KT & Ethics: research lifecycle overview; KT 'last mile'; privacy/security; Indigenous Data Sovereignty (OCAP, CARE); AI hallucinations & bias (AI fluency does not equal correctness).	Explore open data portals; complete Data Intake Card; discuss data formats, provenance, stewardship, and responsible AI use; course workflow overview (Canvas + GitHub).	Required unweighted Data Intake Card checkpoint (no percentage grade)
2 (Sept 22)	Modern Workflows: IDE vs notebook vs script; literate programming; reproducible project structure; Codespaces as the shared lab bench; Quarto basics.	Fork course template repo into personal GitHub account; launch Codespace; tour VS Code; edit + render Quarto notebook; stage, commit, sync; stop Codespace.	Assignment 1 (Mon 4 PM)
3 (Sept 29)	R with AI (Code runs does not equal code is correct): data import/export; core constructs (objects, functions, control flow); the tidyverse pipeline; debugging with LLMs; writing custom functions; documenting project dependencies against the shared devcontainer.	Run R in Codespace; load dataset; inspect & transform with dplyr; write a custom function; document dependencies in Quarto; commit incrementally.	Assignment 2 (Mon 4 PM)
4 (Oct 6)	Git & Collaboration: commits as analysis provenance; branching/merging; Pull Requests & peer review mindset; repo hygiene (.gitignore, relative paths).	Git basics via GUI; meaningful commits; .gitignore & relative paths; create branch, open Pull Request; one member forks group project template, adds teammates as collaborators.	Assignment 3 (Mon 4 PM); Milestone 0: form project group
5 (Oct 13)	Polyglot Awareness & R Deepening: R as the core course language; when Python is useful; reading short Python/pandas examples; translation traps; AI-assisted parity checks; R mastery exercises on dplyr and functions.	Read and run a short Python/pandas notebook in Codespaces; compare with an R/tidyverse solution; use AI to translate a small task; verify parity with checks; complete R deepening exercises.	Assignment 4 (Mon 4 PM)
Part 2: Analysis, Visualization & Dashboards			

(continued)

Week	Seminar Topics	Activities	Deadlines
6 (Oct 20)	Data Visualization: visual reasoning in AI-assisted workflows; steering AI-generated plots (ggplot2, plotly); evaluating clarity, bias, aggregation, and misrepresentation.	Generate & refine visualizations with AI (ggplot2/plotly); evaluate AI-generated plots for clarity, bias, aggregation choices, and misleading design; embed visuals in Quarto with captions.	Assignment 5 (Mon 4 PM); Milestone 1: project proposal
7 (Oct 27)	EDA, Table 1 & AI Auditing: descriptive analysis workflow; what summary statistics claim; auditing AI-generated summaries/plots for planted coding, statistical, and stewardship errors; revisiting data provenance and Indigenous Data Sovereignty.	Perform EDA on project data; create Table 1; complete an AI audit exercise with planted errors; add or revise a data provenance and stewardship statement; checkpoint commits.	Assignment 6 (Mon 4 PM); Milestone 2: Preliminary Analysis
8 (Nov 3)	Dashboard Prototypes for KT: Quarto dashboard-style products and Shiny concepts; building a minimal dashboard from EDA outputs; deployment options introduced as demonstrations only (e.g., Shinylive/GitHub Pages). Note: no class Tue Nov 10 (UBC Mid-Term Break, Nov 9-11); next class is Tue Nov 17.	Build a minimal dashboard-style KT product from EDA outputs; test in Codespaces; view optional Shiny/Shinylive deployment demo; identify which dashboard pathway is feasible for the term project.	Assignment 7 (Mon 4 PM)
9 (Nov 17)	Communication of Scientific Findings: audience-specific KT; plain language summaries; Quarto revealjs slides; citations & references; peer review practices.	Create Quarto revealjs slides (guided intro to slide format); integrate figures; structured peer feedback on clarity & KT.	Assignment 8 (Mon 4 PM); Milestone 3: Project Update draft due Mon Nov 23
	Part 3: Communication, Portfolios & Presentations		
10 (Nov 24)	Writing & Publishing Reports: scientific formatting in Quarto; dynamic reporting; embedding interactive elements; enabling GitHub Pages; publishing your portfolio; citation/grounding audit.	Draft report in Quarto; add citations (Zotero/BibTeX); embed visuals; enable GitHub Pages and publish portfolio site (or private equivalent).	Assignment 9 (Mon 4 PM); Milestone 4: Peer Review due Mon Nov 30

(continued)

Week	Seminar Topics	Activities	Deadlines
11 (Dec 1)	Portfolio Surgery & Optional Advanced Deployment: reproducibility stress-test in a fresh Codespace; fix broken paths, missing dependencies, unclear READMEs; optional demo of AI-generated static HTML/JS dashboards and optional Python-flavoured project pathway. Final class: the term project presentation is a narrated slide deck submitted online with the Final Portfolio (no in-class session).	Run your own project in a fresh Codespace and repair failures (README, paths, dependencies); act on the peer feedback received (M4); record your narrated presentation slide deck for the final portfolio.	(No new submission; portfolio surgery + act on peer feedback in class)
-	-	-	Final Portfolio (with required narrated presentation slide deck) due Dec 11 (Fri 4 PM)

Learning Outcomes

By the end of this course, students will be able to:

1. Explain key ethical, privacy, security, Indigenous Data Sovereignty, and knowledge translation issues in health data science, including responsible use of generative AI tools.
2. Use GitHub Codespaces, Quarto, Git, and GitHub to organize, document, version, and reproduce a small health data analysis.
3. Import, clean, summarize, and visualize health-related data using R and the tidyverse.
4. Read and compare short Python/pandas examples with AI assistance, identify common translation traps between R and Python, and explain when Python may be an appropriate tool.
5. Use generative AI as a coding assistant while critically auditing AI-generated code and outputs for coding errors, hallucinated claims, bias, security risks, and reproducibility problems.
6. Conduct a bounded exploratory data analysis and communicate descriptive findings using clear tables, figures, narrative interpretation, and appropriate limitations.
7. Create a dashboard-style knowledge translation product and a reproducible Quarto report that translate health data findings for a defined audience.
8. Use the shared devcontainer and project dependency documentation so analyses can be rerun in a fresh Codespace.
9. Provide constructive peer feedback on the clarity, reproducibility, auditability, and knowledge translation value of another project.

Learning Activities

Weekly Assignments (Weeks 2-10)

Each week, students complete short applied assignments that reinforce the tools and concepts from class. These assignments are scaffolded and practice-oriented, but they also provide an important measure of steady engagement and skill development. Unless otherwise noted, assignments are due the Monday after class at 4 PM, giving students time to revisit the activity, troubleshoot technical issues, and use office hours or discussion boards.

Grading policy:

- **Assignments 1-3 (Weeks 2-4) are required but worth 0% numerically.** These foundational assignments (Codespace/Quarto, R basics, Git) are graded on a Complete/Incomplete basis. Each must be completed to pass the course. Incomplete submissions may be resubmitted once within one week. A student who completes A1-A3 and otherwise earns at least 50% on the graded components will pass.
- **Assignments 4-9 (Weeks 5-10): Best 4 of 6 count.** Only your best 4 scores count toward your final grade (each worth 8.75%; 35% total). This provides flexibility for difficult weeks - you can drop your two weakest - while ensuring consistent engagement after the foundation is established. Assignment 6 includes a dedicated AI-audit task involving planted errors in AI-generated code, summaries, and/or visualizations.

Reproducibility requirement

All assignments must render/run in a GitHub Codespace without manual intervention. Assignments that violate this rule will be returned for a "Reproducibility Fix" and may incur a grade deduction.

Term Project / Data Science Portfolio

A capstone project synthesized across the full term, broken into milestones:

Milestone	Due	Description
M0: Group Formation	Week 4 (Mon)	Form project group of 3-4 students; one member forks template and adds teammates as collaborators
M1: Proposal	Week 6 (Mon)	Research question, dataset selection, feasibility check, and one-paragraph data provenance/stewardship statement

Milestone	Due	Description
M2: Preliminary Analysis	Week 7 (Mon)	Initial data cleaning, exploratory visualizations, Table 1 or equivalent descriptive summary, runnable repository
M3: Project Update	Mon Nov 23 *(draft)*	Draft dashboard-style KT product + report draft submitted for peer review; polish is expected later in the Final Portfolio
M4: Peer Review	Mon Nov 30	Constructive, specific, actionable feedback on a peer's M3 (reproducibility + auditability checks); feedback is discussed in the Dec 1 class
Presentation (10%)	With Final Portfolio (Dec 11)	Narrated 5-7 slide deck submitted online as a required part of the final portfolio (no in-class session); knowledge translation focus
Final Portfolio (35%)	Dec 11 (Fri)	Complete report, required dashboard-style KT product, narrated presentation deck, and fully auditable code repository

Analytic Scope of the Term Project

The project is graded on the data lifecycle, reproducibility, visualization clarity, AI auditability, and knowledge translation - not on the sophistication of statistical inference. Descriptive summaries, well-chosen visualizations, clear documentation of data decisions, and clearly stated limitations are expected. Formal hypothesis tests, regression models, predictive models, or causal claims are out of scope unless agreed with the instructor in the proposal.
The required knowledge translation product is a small dashboard-style product suitable for the chosen audience. This may be implemented through Quarto dashboard-style reporting, R-generated interactive elements, or another instructor-approved pathway that runs reliably in GitHub Codespaces. Advanced deployment options such as Shiny hosting, Shinylive, or AI-generated HTML/JavaScript may be demonstrated, but are not required for every student.

Reproducibility Contract

The Reproducibility Checklist

A significant portion of the final grade depends on reproducibility. The following checklist serves as both a learning guide and the grading rubric for the Final Portfolio. Refer to it every week:

1. **Fresh Codespace test:** The project launches and runs start-to-finish in a newly created GitHub Codespace without manual intervention.
2. **Relative paths only:** No absolute paths anywhere in the codebase.
3. **Dependencies declared:** R packages are loaded explicitly and named in dependency notes; approved environment changes are recorded in the devcontainer setup or manifest.
4. **No manual steps:** The pipeline from raw data to final output is fully scripted.
5. **README quality:** `README.md` explains the research question, data provenance, how to reproduce the analysis, and known limitations.
6. **.gitignore configured:** Secrets, OS junk, and temporary support files are excluded. Global `*.html` and `*.pdf` rules are not used; every rendered artifact explicitly listed by an assessment is committed with its source.
7. **Commit history:** Incremental, meaningful commits throughout the term.
8. **Documentation:** Code is commented; Quarto documents interleave code with interpretive narrative.
9. **Dashboard-style KT product works:** The project includes a small dashboard-style knowledge translation product that renders or runs in the supported course workflow. If the portfolio is public, it should be viewable through GitHub Pages or another instructor-approved route; if the project is private, the teaching team must be able to render or preview it in a fresh Codespace.

This contract is introduced in Week 1 and reinforced in every subsequent week.

Assessments of Learning

Scope & Expectations

- **Focus:** This project prioritizes descriptive health data analysis, visualization, dashboard-style knowledge translation, reproducible reporting, and AI auditability.
- **Audience:** SPPH 381H is an undergraduate course. No separate graduate-level requirements are assumed unless a graduate section is formally approved. Advanced students may pursue optional extensions with instructor approval, but full credit is achievable through the core R-based pathway.

Component	Weight	Notes
Foundational Assignments (Weeks 2-4)	**Required (0%; Complete/ Incomplete)**	Must complete all 3 to pass; does not contribute to numeric grade
Weekly Assignments (Best 4 of 6, Weeks 5-10)	**35%**	Best 4 counted; each counted assignment worth 8.75%
Milestone 1: Project Proposal	**5%**	Dataset, question, feasibility, provenance/stewardship statement
Milestone 2: Preliminary Analysis	**5%**	Cleaning, EDA, descriptive summary, runnable repo
Milestone 3: Project Update	**5%**	Draft dashboard-style KT product + report draft for peer review
Milestone 4: Peer-Review Contribution	**5%**	Specific, actionable feedback on a peer's work; includes reproducibility check
Presentation (with Final Portfolio)	**10%**	Narrated 5-7 slide deck submitted online as a required part of the final portfolio (Dec 11)
Final Portfolio	**35%**	Report, required dashboard-style KT product, narrated presentation deck, auditable repository; graded against Reproducibility Contract
Total	**100%**	

- **Analytic boundaries:** Students are expected to clearly describe their data, workflow, results, and limitations. Formal modelling, prediction, causal inference, or hypothesis testing is optional only with instructor approval and must be described cautiously.
- **Python option:** A small Python/pandas component may be used for an optional extension if it runs in Codespaces and is documented clearly; the required project can be completed entirely in R.
- **Auditability:** Graded against the Reproducibility Contract (above). The project must run in a fresh Codespace.

Late Submissions

- **Foundational Assignments (1-3):** May be resubmitted once within one week of the original deadline.
- **Weekly Assignments (4-9):** Due Mondays at 4 PM unless otherwise noted. Late submissions are not accepted. Missed assignments count as one of your dropped grades (Best 4 of 6 policy).

- **Final Portfolio:** Late submissions penalized 10% per day. Submissions more than 5 days late not accepted without prior arrangement.
- **Academic Concession:** Requests handled in accordance with UBC Policy V-135. Contact the instructor as soon as the need arises.

Course Resources

Online Textbook (OER)

The course textbook is a free, open-access **Quarto Book** published on GitHub Pages. Each chapter corresponds to one week of the course. The textbook serves as both pre-reading and reference material. It is version-controlled, searchable, and updated each term.
Link will be posted on Canvas before the first class.

Survival Guide & FAQ

A living document maintained on Canvas (and in the course GitHub repository) addressing the most common technical issues: Codespace launch failures, Git merge conflicts, rendering errors, AI tool access, package installation, rendering problems, and Codespaces usage management. Updated weekly by the teaching team based on issues encountered in class.

Required Hardware (BYOD Policy)

This course adopts a **Cloud-First** approach. Because we use GitHub Codespaces, you do not need a high-performance computer. Any device capable of running a modern web browser (including Chromebooks, older laptops, or tablets with a keyboard) is sufficient. GitHub Codespaces is the required and supported computing environment; other platforms (e.g., Colab, Kaggle) are not supported for graded work unless explicitly approved.

Bring your device to every class. Come fully charged, as classroom outlets may be limited.

Need a device? Students can borrow from the UBC Library.

Codespaces: Hours and Best Practices

GitHub Codespaces usage is measured in **core-hours**, not clock hours. At the time this outline was prepared, GitHub Free personal accounts included a monthly quota of Codespaces use; on a typical 2-core Codespace this is approximately **60 hours of active runtime per month**. GitHub may change quotas, so the Canvas Survival Guide will link to the current GitHub documentation and show students how to check their remaining usage.

This is sufficient for the course if managed well:

- **Always stop your Codespace** when you finish working (github.com/codespaces -> three dots -> Stop). Leaving a tab open wastes credits.
- **Reopen existing Codespaces** rather than creating new ones each session.
- **Use the default course machine type** unless instructed otherwise; larger machines consume core-hours faster.
- **If you run out of hours**, contact the teaching team for guidance before adding billing information. Plan your work accordingly.
- **If Codespaces is slow or unavailable** (rare), the Survival Guide includes contingency steps. We will communicate via Canvas if a platform-wide outage affects class.

Open Science & Portfolio Policy

A key goal of this course is to ensure you leave with more than just a grade. By the end of the term, you will have a **Data Science Portfolio** featuring real-world health data analysis - a tangible asset for your CV or LinkedIn, whether you keep the repository private or share it publicly.

- **Default: private.** Project repositories are private by default and shared only with the teaching team. There is no penalty for keeping your work private, and you do not need to request or justify this.
- **Going public (encouraged, never required):** After your project passes the course privacy checklist, we encourage you to make the repository public and, if you wish, publish a portfolio site with GitHub Pages. Be aware that a GitHub Pages site is public even when its source repository is private.
- **Private grading route:** If your project stays private, the teaching team grades it by previewing it in a fresh Codespace or by viewing the rendered HTML committed in the private repository.

University Policies

UBC provides resources to support student learning and to maintain healthy lifestyles but recognizes that sometimes crises arise and so there are additional resources to access including those for survivors of sexual violence. UBC values respect for the person and ideas of all members of the academic community. Harassment and discrimination are not tolerated nor is suppression of academic freedom. UBC provides appropriate accommodation for students with disabilities and for religious observances. UBC values academic honesty and students are expected to acknowledge the ideas generated by others and to uphold the highest academic standards in all of their actions.

Details of the policies and how to access support are available on the UBC Senate website.

Other Course Policies

Plagiarism

Students are expected to review the Student Discipline section of the UBC Calendar and know what constitutes plagiarism and academic misconduct. Such activities are subject to penalty.

Generative AI Policy (Permissive but Regulated)

Generative AI tools (e.g., GitHub Copilot) are **permitted and encouraged** in this course. However, you must adhere to the AI-Augmented Workflow guidelines:

- **Attribution:** Acknowledge the use of AI tools in your submissions (e.g., "Code block X was generated by Copilot and debugged by me").
- **Accountability:** You are fully responsible for any errors, biases, or security flaws in the code you submit. "The AI made a mistake" is not a valid excuse.
- **Verification:** Always audit AI output against official documentation, the course notes, and your own data.
- **Audit trail:** For major assignments and the final portfolio, include a brief AI-use note describing what the tool helped with, what you changed, and how you verified the result. The `ai-use-note.md` file is the graded artifact; the inline `# AI prompt:` / `# Verified:` comments taught in class are the working audit trail you keep while coding and summarize in that note.

Learning Analytics

This course uses Canvas, which captures data about student activity. The instructor plans to use analytics data to: view overall class progress, track students' progress for personalized feedback, review content access statistics, and assess participation.

Copyright

All materials of this course (handouts, slides, assessments, readings, etc.) are the intellectual property of the Course Instructor or licensed for use in this course by the copyright owner. Redistribution without permission constitutes a breach of copyright and may lead to academic discipline.

Recording of class sessions by students is not permitted. Selected recordings by the instructor/TAs will be released on Canvas for viewing outside class. Contact the instructor immediately with any objections.